

BANKING MANAGEMENT SYSTEM

Create a Table Named Customer

create table:

```
create table Customers(  
Customer_ID int primary key ,  
Customer_Name varchar(50) not null,  
Phone_No varchar(15),  
City varchar(50),  
Account_Type varchar(50)  
);
```

```
insert into Customers values  
(1, 'Rahul Sharma', 9876543210, 'Pune', 'Saving'),  
(2, 'Snehal Patil', 9988776655, 'Mumbai', 'Current'),  
(3, 'Aman Varma', 9123456780, 'Nagpur', 'Saving'),  
(4, 'Priya Singh', 9012345678, 'Delhi', 'Current'),  
(5, 'Karan Mehta', 9871203456, 'Hyderabad', 'Saving'),  
(6, 'Neha Joshi', 9988001122, 'Pune', 'Current'),  
(7, 'Rohit Kumar', 9765432109, 'Bangalore', 'Saving'),  
(8, 'Pooja Sharma', 9876540001, 'Chennai', 'Saving'),  
(9, 'Vivek Shah', 9001122334, 'Ahmedabad', 'Current'),  
(10, 'Anjali Verma', 9988771100, 'jaipur', 'Saving');
```

```
select * from Customers;
```

Output:

	Customer_ID	Customer_Name	Phone_No	City	Account_Type
1	1	Rahul Shama	9876543210	Pune	Saving
2	2	Snehal Patil	9988776655	Mumbai	Current
3	3	Aman Vama	9123456780	Nagpur	Saving
4	4	Priya Singh	9012345678	Delhi	Current
5	5	Karan Mehta	9871203456	Hyderabad	Saving
6	6	Neha Joshi	9988001122	Pune	Current
7	7	Rohit Kumar	9765432109	Bangalore	Saving
8	8	Pooja Shama	9876540001	Chennai	Saving
9	9	Vivek Shah	9001122334	Ahmedabad	Current
10	10	Anjali Vema	9988771100	jaipur	Saving

Create a Table Named Accounts

Create table:

```
create table Accounts (  
Account_ID int primary key,  
Customer_ID int,  
Balance decimal(10,2),  
Open_Date date,  
  
Foreign key (Customer_ID)  
References Customers(Customer_ID)  
);
```

```
insert into Accounts values  
(1001,1,55000,'2025-01-10'),  
(1002,2,120000,'2024-11-20'),  
(1003,3,35000,'2025-03-15'),  
(1004,4,98000,'2025-02-01'),  
(1005,5,75000,'2025-01-25'),  
(1006,6,150000,'2024-12-18'),  
(1007,7,42000,'2025-04-10'),  
(1008,8,88000,'2025-05-05'),  
(1009,9,200000,'2024-09-30'),  
(1010,10,67000,'2025-03-22');
```

```
select * from Accounts;
```

Output:

	Account_ID	Customer_ID	Balance	Open_Date
1	1001	1	55000.00	2025-01-10
2	1002	2	120000.00	2024-11-20
3	1003	3	35000.00	2025-03-15
4	1004	4	98000.00	2025-02-01
5	1005	5	75000.00	2025-01-25
6	1006	6	150000.00	2024-12-18
7	1007	7	42000.00	2025-04-10
8	1008	8	88000.00	2025-05-05
9	1009	9	200000.00	2024-09-30
10	1010	10	67000.00	2025-03-22

Create a Table Named Transactions

Create table:

```
create table Transactions(  
Transaction_ID int primary key,  
Account_ID int,  
Transaction_Type varchar(50),  
Amount decimal(10,2),  
Transaction_data date,  
  
Foreign key (Account_ID)  
References Accounts(Account_ID)  
);
```

```
insert into Transactions values  
(1,1001,'Deposit',10000,'2026-06-01'),  
(2,1001,'Withdraw',5000,'2026-06-02'),  
(3,1002,'Deposit',25000,'2026-06-02'),  
(4,1003,'Withdraw',3000,'2026-06-03'),  
(5,1004,'Deposit',15000,'2026-06-04'),  
(6,1005,'Deposit',12000,'2026-06-05'),  
(7,1006,'Withdraw',7000,'2026-06-05'),  
(8,1007,'Deposit',9000,'2026-06-06'),  
(9,1008,'Withdraw',4500,'2026-06-06'),  
(10,1009,'Deposit',30000,'2026-06-06'),  
(11,1010,'Withdraw',2000,'2026-06-07'),  
(12,1002,'Withdraw',10000,'2026-06-08'),  
(13,1003,'Deposit',5000,'2026-06-08'),  
(14,1005,'Withdraw',3500,'2026-06-09'),  
(15,1007,'Deposit',15000,'2026-06-09');
```

```
select * from Transactions;
```

Output:

	Transaction_ID	Account_ID	Transaction_Type	Amount	Transaction_data
1	1	1001	Deposit	10000.00	2026-06-01
2	2	1001	Withdraw	5000.00	2026-06-02
3	3	1002	Deposit	25000.00	2026-06-02
4	4	1003	Withdraw	3000.00	2026-06-03
5	5	1004	Deposit	15000.00	2026-06-04
6	6	1005	Deposit	12000.00	2026-06-05
7	7	1006	Withdraw	7000.00	2026-06-05
8	8	1007	Deposit	9000.00	2026-06-06
9	9	1008	Withdraw	4500.00	2026-06-06
10	10	1009	Deposit	30000.00	2026-06-06
11	11	1010	Withdraw	2000.00	2026-06-07
12	12	1002	Withdraw	10000.00	2026-06-08
13	13	1003	Deposit	5000.00	2026-06-08
14	14	1005	Withdraw	3500.00	2026-06-09
15	15	1007	Deposit	15000.00	2026-06-09

Create a Table Named Accounts

Create table:

```
create table Loans(  
Loan_ID int primary key,  
Customer_ID int,  
Loan_Amount decimal(10,2),  
Loan_Type varchar(50),  
  
Foreign key (Customer_ID)  
References Customers(Customer_ID)  
);
```

```
insert into Loans values  
(1,1,500000,'Home Loan'),  
(2,2,200000,'Car Loan'),  
(3,4,100000,'Education Loan'),  
(4,5,300000,'Business Loan'),  
(5,6,150000,'Personal Loan'),  
(6,8,250000,'Home Loan'),  
(7,9,400000,'Business Loan'),  
(8,10,180000,'Car Loan');
```

```
select * from Loans;
```

Output:

	Loan_ID	Customer_ID	Loan_Amount	Loan_Type
1	1	1	500000.00	Home Loan
2	2	2	200000.00	Car Loan
3	3	4	100000.00	Education Loan
4	4	5	300000.00	Business Loan
5	5	6	150000.00	Personal Loan
6	6	8	250000.00	Home Loan
7	7	9	400000.00	Business Loan
8	8	10	180000.00	Car Loan

Display customer names, account numbers, and account balances using INNER JOIN.

Query:

```
select c.Customer_Name,a.Account_ID,a.Balance
from Customers c inner join Accounts a
on c.Customer_ID = a.Customer_ID;
```

Output:

	Customer_Name	Account_ID	Balance
1	Rahul Sharma	1001	55000.00
2	Snehal Patil	1002	120000.00
3	Aman Vama	1003	35000.00
4	Priya Singh	1004	98000.00
5	Karan Mehta	1005	75000.00
6	Neha Joshi	1006	150000.00
7	Rohit Kumar	1007	42000.00
8	Pooja Shama	1008	88000.00
9	Vivek Shah	1009	200000.00
10	Anjali Verna	1010	67000.00

Find the top 3 customers with the highest account balances.

Query:

```
select top(3) c.Customer_Name,a.Balance
from Customers c inner join Accounts a
on c.Customer_ID = a.Customer_ID
order by a.Balance desc;
```

Output:

	Customer_Name	Balance
1	Vivek Shah	200000.00
2	Neha Joshi	150000.00
3	Snehal Patil	120000.00

Show all customers who have taken loans along with loan amount and loan type.

Query:

```
select c.Customer_Name,l.Loan_Amount,l.Loan_Type
from Customers c inner join Loans l
on c.Customer_ID = l.Customer_ID;
```

Output:

	Customer_Name	Loan_Amount	Loan_Type
1	Rahul Shama	500000.00	Home Loan
2	Snehal Patil	200000.00	Car Loan
3	Priya Singh	100000.00	Education Loan
4	Karan Mehta	300000.00	Business Loan
5	Neha Joshi	150000.00	Personal Loan
6	Pooja Shama	250000.00	Home Loan
7	Vivek Shah	400000.00	Business Loan
8	Anjali Vema	180000.00	Car Loan

Find the total deposited amount and total withdrawn amount separately.

Query:

```
select Transaction_type,
sum(amount) as Total_Amount from Transactions
group by Transaction_Type;
```

Output:

	Transaction_type	Total_Amount
1	Deposit	121000.00
2	Withdraw	35000.00

Display customer-wise total transaction amount using GROUP BY.

Query:

```
select c.Customer_Name ,sum(t.Amount) as Total_Transaction_Amount
from Customers c inner join Accounts a
on c.Customer_ID = a.Customer_ID
inner join Transactions t
on a.Account_ID = t.Account_ID
group by c.Customer_Name;
```

Output:

	Customer_Name	Total_Transaction_Amount
1	Aman Vama	8000.00
2	Anjali Vema	2000.00
3	Karan Mehta	15500.00
4	Neha Joshi	7000.00
5	Pooja Shama	4500.00
6	Priya Singh	15000.00
7	Rahul Shama	15000.00
8	Rohit Kumar	24000.00
9	Snehal Patil	35000.00
10	Vivek Shah	30000.00

Find customers whose balances are greater than the average bank balance.

Query:

```
select c.Customer_Name,a.Balance
from Customers c inner join Accounts a
on c.Customer_ID = a.Customer_ID
where a.Balance > (select avg(balance) from Accounts);
```

Output:

	Customer_Name	Balance
1	Snehal Patil	120000.00
2	Priya Singh	98000.00
3	Neha Joshi	150000.00
4	Vivek Shah	200000.00

Show the highest transaction amount performed by each customer.

Query:

```
select c.Customer_Name,max(t.amount) as Highest_transaction_amount
from Customers c inner join Accounts a
on c.Customer_ID = a.Customer_ID
inner join Transactions t
on a.Account_ID = t.Account_ID
group by c.Customer_Name;
```

Output:

	Customer_Name	Highest_transaction_amount
1	Aman Vama	5000.00
2	Anjali Verma	2000.00
3	Karan Mehta	12000.00
4	Neha Joshi	7000.00
5	Pooja Shama	4500.00
6	Priya Singh	15000.00
7	Rahul Shama	10000.00
8	Rohit Kumar	15000.00
9	Snehal Patil	25000.00
10	Vivek Shah	30000.00

Display all customers who have not taken any loans using LEFT JOIN.

Query:

```
select c.Customer_Name,l.Loan_Amount
from Customers c left join Loans l
on c.Customer_ID = l.Customer_ID
where Loan_Amount is null;
```

Output:

	Customer_Name	Loan_Amount
1	Aman Vama	NULL
2	Rohit Kumar	NULL

Find the total number of transactions performed by each customer.

Query:

```
select c.Customer_Name,count(*) as total_transaction
from Customers c inner join Accounts a
on c.Customer_ID = a.Customer_ID
inner join Transactions t
on a.Account_ID = t.Account_ID
group by c.Customer_Name;
```

Output:

	Customer_Name	total_transaction
1	Aman Vama	2
2	Anjali Vema	1
3	Karan Mehta	2
4	Neha Joshi	1
5	Pooja Shama	1
6	Priya Singh	1
7	Rahul Shama	2
8	Rohit Kumar	2
9	Snehal Patil	2
10	Vivek Shah	1

Rank customers based on their account balances using RANK() window function.

Query:

```
select c.Customer_Name,a.Balance,
rank() over(order by balance desc) as ranks
from Customers c inner join Accounts a
on c.Customer_ID = a.Customer_ID;
```

Output:

	Customer_Name	Balance	ranks
1	Vivek Shah	200000.00	1
2	Neha Joshi	150000.00	2
3	Snehal Patil	120000.00	3
4	Priya Singh	98000.00	4
5	Pooja Shama	88000.00	5
6	Karan Mehta	75000.00	6
7	Anjali Vema	67000.00	7
8	Rahul Shama	55000.00	8
9	Rohit Kumar	42000.00	9
10	Aman Vama	35000.00	10

Display dense ranking of customers according to balance using DENSE_RANK().

Query:

```
select c.Customer_Name,a.Balance,
dense_rank() over(order by balance desc) as dense_ranking
from Customers c inner join Accounts a
on c.Customer_ID = a.Customer_ID;
```

Output:

	Customer_Name	Balance	dense_ranking
1	Vivek Shah	200000.00	1
2	Neha Joshi	150000.00	2
3	Snehal Patil	120000.00	3
4	Priya Singh	98000.00	4
5	Pooja Shama	88000.00	5
6	Karan Mehta	75000.00	6
7	Anjali Verna	67000.00	7
8	Rahul Shama	55000.00	8
9	Rohit Kumar	42000.00	9
10	Aman Vama	35000.00	10

Show previous transaction amount using LAG() function.

Query:

```
select c.Customer_Name,t.Amount,
lag(Amount) over(order by amount asc) as previous_transaction_amount
from Customers c inner join Accounts a
on c.Customer_ID = a.Customer_ID
inner join Transactions t
on a.Account_ID = t.Account_ID;
```

Output:

	Customer_Name	Amount	previous_transaction_amount
1	Anjali Verna	2000.00	NULL
2	Aman Vama	3000.00	2000.00
3	Karan Mehta	3500.00	3000.00
4	Pooja Shama	4500.00	3500.00
5	Aman Vama	5000.00	4500.00
6	Rahul Shama	5000.00	5000.00
7	Neha Joshi	7000.00	5000.00
8	Rohit Kumar	9000.00	7000.00
9	Rahul Shama	10000.00	9000.00
10	Snehal Patil	10000.00	10000.00
11	Karan Mehta	12000.00	10000.00
12	Priya Singh	15000.00	12000.00
13	Rohit Kumar	15000.00	15000.00
14	Snehal Patil	25000.00	15000.00
15	Vivek Shah	30000.00	25000.00

Show next transaction amount using LEAD() function.

Query:

```
select c.Customer_Name,t.Amount,  
lead(Amount) over(order by amount asc) as next_transaction_amount  
from Customers c inner join Accounts a  
on c.Customer_ID = a.Customer_ID  
inner join Transactions t  
on a.Account_ID = t.Account_ID;
```

Output:

	Customer_Name	Amount	next_transaction_amount
1	Anjali Verna	2000.00	3000.00
2	Aman Varna	3000.00	3500.00
3	Karan Mehta	3500.00	4500.00
4	Pooja Shama	4500.00	5000.00
5	Aman Varna	5000.00	5000.00
6	Rahul Sharma	5000.00	7000.00
7	Neha Joshi	7000.00	9000.00
8	Rohit Kumar	9000.00	10000.00
9	Rahul Sharma	10000.00	10000.00
10	Snehal Patil	10000.00	12000.00
11	Karan Mehta	12000.00	15000.00
12	Priya Singh	15000.00	15000.00
13	Rohit Kumar	15000.00	25000.00
14	Snehal Patil	25000.00	30000.00
15	Vivek Shah	30000.00	NULL

Calculate running total of transaction amounts using SUM() OVER().

Query:

```
select c.Customer_Name,t.Amount,  
sum(Amount) over(order by amount asc) as Runnig_total_amount  
from Customers c inner join Accounts a  
on c.Customer_ID = a.Customer_ID  
inner join Transactions t  
on a.Account_ID = t.Account_ID;
```

Output:

	Customer_Name	Amount	Runnig_total_amount
1	Anjali Verna	2000.00	2000.00
2	Aman Vama	3000.00	5000.00
3	Karan Mehta	3500.00	8500.00
4	Pooja Shama	4500.00	13000.00
5	Aman Vama	5000.00	23000.00
6	Rahul Shama	5000.00	23000.00
7	Neha Joshi	7000.00	30000.00
8	Rohit Kumar	9000.00	39000.00
9	Rahul Shama	10000.00	59000.00
10	Snehal Patil	10000.00	59000.00
11	Karan Mehta	12000.00	71000.00
12	Priya Singh	15000.00	101000.00
13	Rohit Kumar	15000.00	101000.00
14	Snehal Patil	25000.00	126000.00
15	Vivek Shah	30000.00	156000.00

Find the second highest account balance using subquery .

Query:

```
select max(a.balance) as secound_highest_balance  
from Customers c inner join Accounts a  
on c.Customer_ID = a.Customer_ID  
where a.balance < ( select max(Balance) from Accounts);
```

Output:

	secound_highest_balance
1	150000.00

Find the second highest account balance using window function.

Query:

```
with high_balance as(select c.Customer_Name,a.Balance,
dense_rank() over(order by balance desc) as dense_ranking
from Customers c inner join Accounts a
on c.Customer_ID = a.Customer_ID)
select * from high_balance
where dense_ranking = 2;
```

Output:

	Customer_Name	Balance	dense_ranking
1	Neha Joshi	150000.00	2

Find customers who performed more than 2 transactions.

Query:

```
select c.Customer_Name,count(*) as total_transaction_number
from Customers c inner join Accounts a
on c.Customer_ID = a.Customer_ID
inner join Transactions t
on a.Account_ID = t.Account_ID
group by c.Customer_Name
having count(*)>1
order by c.Customer_Name desc;
```

Output:

	Customer_Name	total_transaction_number
1	Snehal Patil	2
2	Rohit Kumar	2
3	Rahul Shama	2
4	Karan Mehta	2
5	Aman Vama	2

Display customer-wise minimum and maximum transaction amounts.

Query:

```
select c.Customer_Name,  
min(amount) as minimum_transaction_amount,  
max(amount) as maximum_transaction_amount  
from Customers c inner join Accounts a  
on c.Customer_ID = a.Customer_ID  
inner join Transactions t  
on a.Account_ID = t.Account_ID  
group by c.Customer_Name;
```

Output:

	Customer_Name	minimum_transaction_amount	maximum_transaction_amount
1	Aman Vama	3000.00	5000.00
2	Anjali Vema	2000.00	2000.00
3	Karan Mehta	3500.00	12000.00
4	Neha Joshi	7000.00	7000.00
5	Pooja Shama	4500.00	4500.00
6	Priya Singh	15000.00	15000.00
7	Rahul Shama	5000.00	10000.00
8	Rohit Kumar	9000.00	15000.00
9	Snehal Patil	10000.00	25000.00
10	Vivek Shah	30000.00	30000.00